

**SMART CAMPUS ENERGY MONITORING AND  
ENERGY-SAVING RECOMMENDATION SYSTEM USING C**

**CSA0204 – C Programming**

**Assignment Report**

|                          |                                                                                                                     |
|--------------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>Department</b>        | Computer Science and Engineering                                                                                    |
| <b>Team Number</b>       | TEAM 2                                                                                                              |
| <b>Team Members</b>      | Divyadharshini P (192312610)<br>Dhivya R (192312619)<br>Yuvasri S (192511202)<br>Flarance Flomina Ganga (192524347) |
| <b>Faculty</b>           | DR. Selvakumar A                                                                                                    |
| <b>Submission Date</b>   | 03 September 2026                                                                                                   |
| <b>GitHub Repository</b> | <a href="https://github.com/Divi0408/C-Programming">https://github.com/Divi0408/C-Programming</a>                   |

## 1. Title

Smart Campus Energy Monitoring and Energy-Saving Recommendation System Using C

## 2. Problem Statement

An engineering college operates multiple facilities such as academic blocks, computer laboratories, libraries, hostels, canteens, administrative offices, and research laboratories, each consuming significant amounts of electrical energy on a daily basis. The proposed system collects daily energy consumption data for different campus facilities over seven days and analyses the data to determine total and average consumption, highest and lowest consumption days, facilities exceeding permitted energy limits, abnormal consumption patterns, and the top energy-consuming facilities.

The system also calculates potential energy savings for a specified reduction target, generates a sustainability score for each facility, and provides energy-saving recommendations. The implementation uses structures, arrays, strings, functions, pointers, recursion, searching, sorting, decision-making, looping, and file handling. It produces a sustainability report to support campus administrators in reducing energy wastage and promoting responsible consumption. The assignment briefly maps the problem to SDG 7, SDG 11, SDG 12 and SDG 13.

fileciteturn0file1L17-L32

## 3. Objectives

- Collect and store seven days of electricity consumption for multiple campus facilities.
- Calculate weekly totals, daily averages, highest-use and lowest-use days.
- Classify facilities as normal, exceeding the permitted limit, or showing an abnormal peak.
- Identify and rank the top energy-consuming facilities using sorting.
- Search facility records by full or partial facility name.
- Merge additional facility records while updating duplicate records instead of creating duplicates.
- Estimate potential energy savings using a 10% reduction target.

## 4. Requirements and Rubric Alignment

CO2: Examine and write solutions for problems using operators, decision-making, and looping constructs.

CO3: Solve problems for searching, sorting, and merging using array and string processing techniques.

CO5: Design modular and scalable C programs using functions, recursion, pointers, and file handling.

## 5. System Design

### 5.1 Data Structure

| Rubric area                            | Weight | Implementation in this project                                      | Evidence           |
|----------------------------------------|--------|---------------------------------------------------------------------|--------------------|
| Operators & control structures (CO2)   | 10     | if/else classification, switch menu, for loops, validation loops    | Code + tests       |
| Arrays, strings & resource data (CO3)  | 10     | Facility structures, 7-day arrays, character strings and validation | Source code        |
| Searching, sorting & merging (CO3)     | 15     | Partial-name search, bubble sort by total, duplicate-aware merge    | Menu/tests         |
| Functions, recursion & pointers (CO5)  | 15     | Meaningful functions, recursive_total(), pointer parameters         | Source code        |
| File handling, analysis & report (CO5) | 20     | Save/load text files and generated sustainability_report.txt        | Files + output     |
| Menu, validation & correctness         | 20     | Menu-driven workflow and input range/type validation                | Invalid-input test |
| Reflection, design, SDG relevance      | 10     | Design decisions, challenges, limitations, improvements and SDGs    | Reflection section |

Each facility is represented using a Facility structure containing the facility name, facility type, permitted weekly limit, and a seven-element energy array. A global array stores up to 20 facility records.

Key structure:

```
typedef struct { char name[60]; char type[40]; double permitted_limit; double energy[7]; }  
Facility;
```

## 5.2 Processing Workflow

1. Start the program and display the menu.
2. Enter records manually or load the supplied sample dataset.
3. Validate names, limits and seven daily energy values.
4. Store records in the Facility array.
5. Analyse totals and averages; determine daily campus maxima/minima.
6. Classify facility status and calculate sustainability score and savings.
7. Search, sort or merge records as requested.
8. Save/retrieve data through a text file.
9. Generate a sustainability report containing analysis and recommendations.
10. Exit after completing required operations.

## 6. Algorithms / Pseudocode

### 6.1 Overall System Algorithm

BEGIN

Initialize facility\_count = 0.

Display menu.

Read a validated menu choice.

If choice = 1, collect one facility and seven daily readings.

If choice = 2, display all stored records.

If choice = 3, calculate totals, averages, daily campus usage, maxima/minima and status.

If choice = 4, search for a full/partial facility name.

If choice = 5, sort facilities by weekly total in descending order.

If choice = 6, read merge\_data.txt; update matching names and append new names.

If choice = 7, write all records to energy\_records.txt.

If choice = 8, read records from energy\_records.txt.

If choice = 9, generate sustainability\_report.txt with scores and recommendations.

If choice = 10, load the built-in sample dataset.

Repeat until choice = 0.

END

### 6.2 Searching Algorithm

Input search key.

Use substring matching to support partial-name search.

If a match is found, display the record and its analysis status.

If no match is found, report that the facility was not found.

### 6.3 Sorting Algorithm

Compare adjacent facilities using their seven-day total.

If the earlier total is smaller, swap the two Facility structures.

Repeat passes until the list is in descending order.

Display the sorted list as the top-consumer ranking.

### 6.4 Merging Algorithm

Open merge\_data.txt.

For each incoming record, search the current array for the same facility name.

If found, replace the existing record with the incoming record (duplicate update).

If not found and capacity remains, append the new facility.

Count added and updated records.

Close the file and display the merge summary

### 6.5 Resource Analysis Algorithm

For each facility, recursively calculate the seven-day total.

Compute average = total / 7.

Find highest and lowest day by scanning the seven daily values.

If weekly total > permitted limit, classify as EXCEEDS LIMIT.

Else if highest daily value >  $1.30 \times$  average, classify as ABNORMAL PEAK.

Otherwise classify as NORMAL.

Calculate a sustainability score from the consumption-to-limit ratio.

Generate a recommendation according to the detected condition.

## 7. Complete C Source Code

The following is the complete source code used for the demonstration. It is intentionally included in text format so that it can be copied to a .c file and compiled. The program uses the requested C concepts and a menu-driven interface.

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#define MAX 20
#define DAYS 7

typedef struct {
    char name[50], type[30];
    float limit, e[DAYS];
} Facility;

float total(float e[], int n) {
    if (n == 0)
        return 0;
    return e[n - 1] + total(e, n - 1);
}

int find(Facility f[], int n, char key[]) {
    for (int i = 0; i < n; i++)
        if (strstr(f[i].name, key))
            return i;
    return -1;
}

void input(Facility *f) {
    printf("Facility name: ");
```

```

scanf(" %[^\\n]", f->name);
printf("Facility type: ");
scanf(" %[^\\n]", f->type);
printf("Permitted weekly limit: ");
scanf("%f", &f->limit);
for (int i = 0; i < DAYS; i++) {
    printf("Day %d energy: ", i + 1);
    scanf("%f", &f->e[i]);
}
}

void display(Facility *f) {
    float t = total(f->e, DAYS);
    printf("\\n%s (%s)\\n", f->name, f->type);
    printf("Weekly: %.2f kWh | Average: %.2f kWh\\n",
        t, t / DAYS);
    printf("Daily: ");
    for (int i = 0; i < DAYS; i++)
        printf("%.1f ", f->e[i]);
    if (t > f->limit)
        printf("\\nStatus: EXCEEDS LIMIT");
    else {
        int high = 0;
        for (int i = 1; i < DAYS; i++)
            if (f->e[i] > f->e[high])
                high = i;
        if (f->e[high] > 1.3 * t / DAYS)
            printf("\\nStatus: ABNORMAL PEAK");
    }
}

```



```

        else

            printf("\nStatus: NORMAL");

        }

        printf("\nPotential 10%% saving: %.2f kWh\n", t * 0.10);
    }

void analyse(Facility f[], int n) {

    float day[DAYS] = {0};

    for (int i = 0; i < n; i++)

        for (int j = 0; j < DAYS; j++)

            day[j] += f[i].e[j];

    int high = 0, low = 0;

    for (int i = 1; i < DAYS; i++) {

        if (day[i] > day[high]) high = i;

        if (day[i] < day[low]) low = i;

    }

    printf("\nTotal weekly campus energy: %.2f kWh\n",

        total(day, DAYS));

    printf("Highest day: Day %d (%.2f kWh)\n",

        high + 1, day[high]);

    printf("Lowest day: Day %d (%.2f kWh)\n",

        low + 1, day[low]);

    for (int i = 0; i < n; i++) {

        float t = total(f[i].e, DAYS);

        float score = 100 - (t / f[i].limit) * 100;

        if (score < 0) score = 0;

        printf("%-20s Total: %7.2f Score: %.1f\n",

            f[i].name, t, score);
    }
}

```

```

    }
}

void sort(Facility f[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (total(f[j].e, DAYS) < total(f[j + 1].e, DAYS)) {
                Facility temp = f[j];
                f[j] = f[j + 1];
                f[j + 1] = temp;
            }

    printf("\nTop energy consumers:\n");
    for (int i = 0; i < n; i++)
        printf("%d. %s - %.2f kWh\n",
            i + 1, f[i].name, total(f[i].e, DAYS));
}

void save(Facility f[], int n) {
    FILE *fp = fopen("energy_records.txt", "w");

    if (!fp) {
        printf("File error.\n");
        return;
    }

    for (int i = 0; i < n; i++) {
        fprintf(fp, "%s,%s,%.2f",
            f[i].name, f[i].type, f[i].limit);

        for (int j = 0; j < DAYS; j++)

```

```

        fprintf(fp, " ,%.2f", f[i].e[j]);
        fprintf(fp, "\n");
    }
    fclose(fp);
    printf("Records saved successfully.\n");
}

void merge(Facility f[], int *n) {
    FILE *fp = fopen("merge_data.txt", "r");
    Facility x;
    int i, found;
    if (!fp) {
        printf("Merge file not found.\n");
        return;    }
    while (fscanf(fp, " %49[^\n],%29[^\n],%f",
        x.name, x.type, &x.limit) == 3) {
        for (i = 0; i < DAYS; i++)
            fscanf(fp, "%f", &x.e[i]);
        found = find(f, *n, x.name);
        if (found >= 0)
            f[found] = x;
        else if (*n < MAX)
            f[(*n)++] = x; }
    fclose(fp);
    printf("Records merged successfully.\n");}

int main() {
    Facility f[MAX];
    int n = 0, ch, i, k;

```

```

char key[50];

do {
    printf("\n\n===== SMART CAMPUS ENERGY SYSTEM =====\n");
    printf("1. Add Facility\n");
    printf("2. Display Facilities\n");
    printf("3. Analyse Energy\n");
    printf("4. Search Facility\n");
    printf("5. Sort by Consumption\n");
    printf("6. Merge Records\n");
    printf("7. Save Records\n");
    printf("0. Exit\n");
    printf("Choice: ");
    scanf("%d", &ch);
    switch (ch) {
    case 1:
        if (n < MAX) {
            input(&f[n]);
            n++;
        } else
            printf("Maximum facilities reached.\n");
        break;
    case 2:
        for (i = 0; i < n; i++)
            display(&f[i]);
        break;
    case 3:
        analyse(f, n);

```

```
        break;
    case 4:
        printf("Enter facility name: ");
        scanf("%s", key);
        k = find(f, n, key);
        if (k >= 0)
            display(&f[k]);
        else
            printf("Facility not found.\n");
        break;
    case 5:
        sort(f, n);
        break;
    case 6:
        merge(f, &n);
        break;
    case 7:
        save(f, n);
        break;
    case 0:
        printf("Program ended.\n");
        break;
    default
        printf("Invalid choice.\n");
    }
}
while (ch != 0);
return 0;
```

## 8. Test Cases and Validation

The test plan covers normal, boundary/edge and invalid-input cases, together with search, sorting, merging, classification and file storage/retrieval as required by the assignment deliverables. fileciteturn0file1L298-L309

| ID   | Test                          | Input             | Expected/Observed                                         | Status |
|------|-------------------------------|-------------------|-----------------------------------------------------------|--------|
| TC01 | Load sample data              | Menu 10           | 5 facilities loaded                                       | PASS   |
| TC02 | Weekly total / daily analysis | Menu 3            | Totals, averages, highest/lowest day and status displayed | PASS   |
| TC03 | Search existing facility      | Search 'Research' | Research Laboratory found                                 | PASS   |
| TC04 | Sort consumers                | Menu 5            | Facilities ranked by weekly total descending              | PASS   |
| TC05 | Merge duplicate + new         | Menu 6            | Academic Block A updated; Seminar Hall added              | PASS   |
| TC06 | File save                     | Menu 7            | Records written to energy_records.txt                     | PASS   |
| TC07 | Report generation             | Menu 9            | sustainability_report.txt created                         | PASS   |
| TC08 | Invalid menu input            | Menu 'abc'        | Program rejects input and asks again                      | PASS   |
| TC09 | Zero/negative energy          | Enter -10         | Validation rejects value                                  | PASS   |
| TC10 | Upper boundary                | Enter 1,000,000   | Accepted as maximum allowed daily/limit input             | PASS   |

## 9. Result / Output Screenshot

A representative execution was run using the built-in sample dataset. The execution demonstrates analysis, search, sorting, merging, file saving/report generation and invalid-input validation.

```
SMART CAMPUS ENERGY ANALYSIS
Facilities monitored : 5
Total weekly energy : 8055.00 kWh
Campus daily average : 1150.71 kWh
Highest consumption day: Day 5 (1375.00 kWh)
Lowest consumption day : Day 1 (1060.00 kWh)
FACILITY STATUS
Academic Block A      1730.00      NORMAL      55.2      173.00
Computer Lab 1       1545.00      EXCEEDS LIMIT 48.5      154.50
Central Library       1155.00      NORMAL      55.0      115.50
Boys Hostel           2065.00      EXCEEDS LIMIT 48.4      206.50
Research Laboratory   1560.00      ABNORMAL PEAK 53.3      156.00
Enter choice: Enter full or partial facility name: Found: Research Laboratory | Research | Weekly 1560.00 kWh
Enter choice: Merge complete: 1 record(s) added, 1 duplicate record(s) updated.
Enter choice: Saved 6 record(s) to energy_records.txt.
Enter choice: Sustainability report generated: sustainability_report.txt
Academic Block A      Academic      1900.0 250 255 265 260 255 250 245
Computer Lab 1       Laboratory    1500.0 210 220 215 225 230 225 220
Central Library       Library      1200.0 150 155 160 165 170 175 180
Boys Hostel           Hostel       2000.0 280 290 300 310 305 295 285
Research Laboratory   Research     1600.0 180 190 185 195 420 200 190
Enter choice: Invalid input. Enter a value from 0 to 10.
```

Figure 1. Representative console output from the C program.

### 9.1 Sample Validation Summary

| Facility            | Weekly kWh | Average kWh/day | Classification |
|---------------------|------------|-----------------|----------------|
| Academic Block A    | 1730       | 247.14          | NORMAL         |
| Computer Lab 1      | 1555       | 222.14          | EXCEEDS LIMIT  |
| Central Library     | 1155       | 165.00          | NORMAL         |
| Boys Hostel         | 2070       | 295.71          | EXCEEDS LIMIT  |
| Research Laboratory | 1560       | 222.86          | ABNORMAL PEAK  |

## 10. Reflection / Evaluation

### 10.1 Design Decisions

A structure was selected because every facility has multiple related attributes: identity, type, permitted limit and seven daily readings. Arrays make the seven-day processing direct, while functions separate input, analysis, searching, sorting, merging, file operations and reporting. A recursive function is used for the weekly total to demonstrate recursion without making the main workflow dependent on recursion. Pointer parameters are used when functions need to update or operate directly on facility records.

### 10.2 Challenges

The main implementation challenges are maintaining consistent record formats during file operations, handling duplicate facilities during merging, validating numeric input, and ensuring that the classification logic remains deterministic. These are addressed through bounded input functions, explicit duplicate search, comma-separated file records and separate analysis functions.

### 10.3 Limitations

- The current program uses a fixed maximum of 20 facilities.
- The merge and save files use simple comma-separated text and therefore do not support commas inside names/types.
- The abnormal-peak threshold is fixed at 130% of the facility's average daily consumption.
- The sustainability score is rule-based and is not calibrated against actual campus utility benchmarks.
- The current version does not connect to live smart-meter/IoT data.



## 10.4 Future Enhancements

- Connect the program to live smart meters or IoT gateways.
- Store larger datasets using a database rather than a fixed-size array and text files.
- Add graphical dashboards and monthly/yearly trend analysis.
- Allow administrators to configure thresholds and reduction targets.
- Add role-based access and audit logs for changes to energy records.

## 10.5 SDG Relevance

The assignment brief explicitly maps the solution to SDG 7 (Affordable and Clean Energy), SDG 11 (Sustainable Cities and Communities), SDG 12 (Responsible Consumption and Production), and SDG 13 (Climate Action). The system contributes to these themes by making energy consumption measurable, identifying excessive use, estimating possible savings, and recommending actions that reduce wastage. fileciteturn0file1L8-L12

## 11. Individual Contribution – One Page Write-up

### 11.1 Individual Contribution – Student 1

|                             |                                                               |
|-----------------------------|---------------------------------------------------------------|
| <b>Student Name</b>         | Divyadharshini P                                              |
| <b>Register Number</b>      | 192312610                                                     |
| <b>Team Number</b>          | 2                                                             |
| <b>Primary contribution</b> | Facility data structure, input handling and data entry module |

#### One-page reflection:

I contributed to the Smart Campus Energy Monitoring and Energy-Saving Recommendation System by working mainly on the design and management of facility data. I helped organize the information required for each campus facility, including the facility name, facility type, permitted weekly energy limit and daily energy consumption for seven days. I worked with the C programming concepts of structures, arrays, strings, functions, loops and conditional statements to store and process the facility information efficiently. I also contributed to the input and display sections of the program and checked whether the entered energy values were correctly stored and displayed. During testing, I used different facility details and energy consumption values to verify the correctness of the program. I checked normal values, low consumption values and values close to the permitted limit. I also participated in identifying and correcting errors during the development of the program. Through this project, I gained a better understanding of how structures and arrays can be used together to represent real-world data in C. I also improved my understanding of modular programming and learned how individual functions can work together as part of a larger application. The project helped me understand how programming can be applied to a practical campus energy-management problem. By recording and monitoring energy consumption, the system can help identify areas where electricity is being unnecessarily consumed and support energy-saving practices. This contributes to the sustainability objectives of the project and its relevance to SDG 7, SDG 11, SDG 12 and SDG 13. Overall, my contribution helped establish the basic data-management and input components required for the successful operation of the Smart Campus Energy Monitoring System.

## 11.2 Individual Contribution – Student 2

|                             |                                                                              |
|-----------------------------|------------------------------------------------------------------------------|
| <b>Student Name</b>         | Dhivya R                                                                     |
| <b>Register Number</b>      | 192312619                                                                    |
| <b>Team Number</b>          | 2                                                                            |
| <b>Primary contribution</b> | Energy consumption analysis, calculations and facility status classification |

### One-page reflection:

My contribution to the Smart Campus Energy Monitoring and Energy-Saving Recommendation System focused mainly on the analysis of energy consumption data. I worked on calculating the total weekly energy consumption, daily average consumption and identifying the highest and lowest consumption days for each facility. I also contributed to the logic used to determine the status of a facility based on its energy usage. The program can identify whether a facility is operating within the permitted limit, exceeding the permitted limit or showing an abnormal peak in consumption. I worked with important C programming concepts such as functions, arrays, recursion, arithmetic operations, loops and decision-making statements. The recursive calculation function was used to process the daily energy values and determine the total consumption for a facility. I also participated in testing the analysis functions using different sets of energy readings to make sure that the calculated totals, averages and facility status were correct. Special attention was given to cases where the weekly consumption exceeded the permitted limit or where one particular day had unusually high energy consumption. Through this project, I developed a better understanding of how programming logic and mathematical calculations can be combined to analyse real-world data. I also learned how suitable conditions and thresholds can be used to identify abnormal consumption patterns. The analysis performed by the system can help campus management identify facilities that require attention and suggest possible energy-saving measures. This supports the sustainability purpose of the project and contributes to SDG 7, SDG 11, SDG 12 and SDG 13. Overall, my contribution helped the team implement the energy-analysis part of the system and improved my practical knowledge of applying C programming concepts to a real-world problem.

### 11.3 Individual Contribution – Student 3

|                             |                                                           |
|-----------------------------|-----------------------------------------------------------|
| <b>Student Name</b>         | Yuvasri S                                                 |
| <b>Register Number</b>      | 192511202                                                 |
| <b>Team Number</b>          | 2                                                         |
| <b>Primary contribution</b> | Searching, sorting and merging of facility energy records |

#### **One-page reflection:**

I contributed to the Smart Campus Energy Monitoring and Energy-Saving Recommendation System by working mainly on searching, sorting and managing facility records. I helped implement the search functionality, which allows users to find a particular facility by entering its name or a relevant part of its name. I also contributed to the sorting operation, which arranges facilities according to their weekly energy consumption so that high energy-consuming facilities can be identified easily. Another important part of my contribution was the merging of additional facility records with the existing records. The merge functionality allows new information to be added and existing facility information to be updated when a matching record is found. While working on these features, I used C programming concepts such as arrays of structures, strings, functions, loops, comparisons and pointers. I tested the search functionality using both matching and non-matching facility names and checked whether the correct facility information was displayed. I also tested the sorting operation with different energy consumption values to ensure that the records were arranged correctly. For the merge functionality, I considered both new records and records that already existed in the system. These activities helped me understand how data can be searched, organized and updated efficiently using C. I also learned how different algorithms and data structures can be combined within a single menu-driven application. The project demonstrated how energy information can be organized to help users quickly identify facilities with high consumption and focus on possible energy-saving actions. This supports the sustainability objectives of the project and its connection with SDG 7, SDG 11, SDG 12 and SDG 13. Overall, my contribution focused on improving the usability and data-management capabilities of the system.

#### 11.4 Individual Contribution – Student 4

|                             |                                                                   |
|-----------------------------|-------------------------------------------------------------------|
| <b>Student Name</b>         | Flarance Flomina Ganga                                            |
| <b>Register Number</b>      | 192524357                                                         |
| <b>Team Number</b>          | 2                                                                 |
| <b>Primary contribution</b> | File handling, report generation, testing and program integration |

##### **One-page reflection:**

My contribution to the Smart Campus Energy Monitoring and Energy-Saving Recommendation System focused mainly on file handling, report generation, testing and integration of the different components. I worked with C file-handling concepts to store facility energy records and retrieve them when required. The program uses files to maintain energy records and also supports merging additional records from an external file. I contributed to the generation of the sustainability report, which presents useful information such as weekly energy consumption, average consumption, facility status, sustainability score, potential energy savings and recommendations. I also participated in integrating the different functions into the menu-driven program so that the user can perform operations such as adding facilities, displaying records, analysing energy consumption, searching, sorting, merging and saving records. During testing, I checked the different menu options and verified that the program produced the expected results. I also tested file operations to ensure that records could be stored and retrieved correctly and that the sustainability report was generated successfully. Through this project, I improved my understanding of file pointers, file opening and closing, formatted input and output, functions and modular programming in C. I also gained experience in debugging and integrating different modules developed as part of a team project. The system demonstrates how software can support responsible energy consumption by monitoring electricity usage and providing useful recommendations for reducing wastage. This contributes to the sustainability goals associated with the project, particularly SDG 7, SDG 11, SDG 12 and SDG 13. Overall, my contribution helped connect the individual program components, handle persistent data and produce meaningful output from the energy-monitoring system